

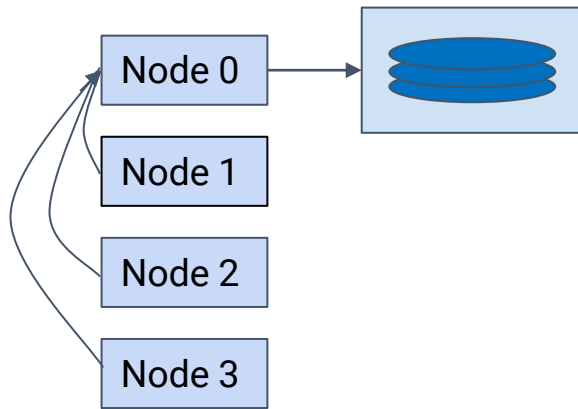
Файлы и структуры в MPI

Работа с файлами

- Как показывает практика, результатом работы многих вычислительных приложений являются файлы с данными.
- Запись файла является наиболее затратной операцией для вычислительной системы.
- При работе в рамках технологий параллельного программирования ситуация усугубляется необходимостью правильно организовать доступ к файлу.

Последовательная работа с файлами

- I/O поддерживается только на определенном узле многопроцессорной системы
- В программе используется высокоуровневая библиотека, не поддерживающая параллельный ввод-вывод
- Результирующий файл должен обрабатываться последовательным ПО
- Возможное повышение эффективности за счет буферизации данных



4 nodes, 1 file

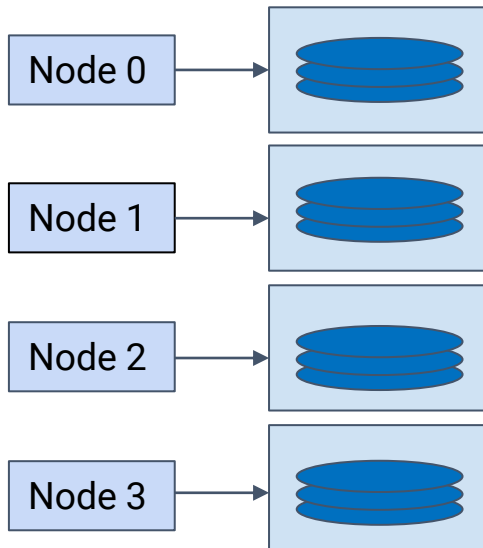
Параллельная работа с множеством файлов

Преимущества:

- Параллельный доступ

Недостатки:

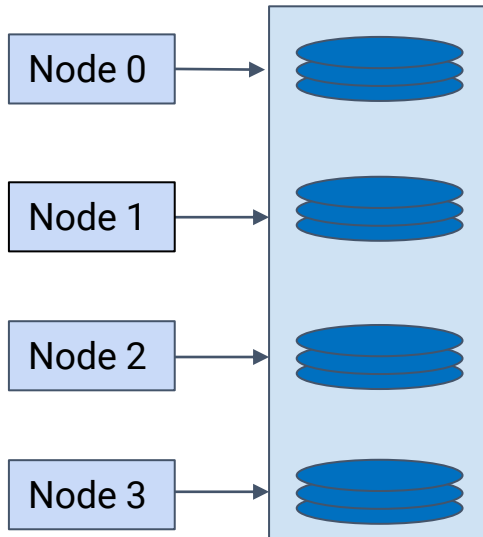
- Много файлов
- Использование файлов при повторном запуске (то же число процессов)



4 nodes, 4 file

Параллельная работа с одним файлом

- Параллельный доступ к файлу
- Запись происходит в один файл
- Работоспособность при различном числе параллельных процессов



4 nodes, 1 logical file

MPI-I/O: Базовый алгоритм работы

- Определение необходимых переменных и типов данных
- Открытие файла ([MPI_File_open](#))
- Установка вида файла ([MPI_File_set_view](#))
- Запись/чтение ([MPI_File_write](#), [MPI_File_read](#))
- Для неблокирующих операций, ожидание их завершения (напр., [MPI_Wait](#))
- Заккрытие файла ([MPI_File_close](#))

MPI-I/O: Открытие файла

```
int MPI_File_open(MPI_Comm comm, char *filename,  
int amode, MPI_Info info, MPI_File *fh)
```

- IN `comm` коммуникатор (дескриптор)
- IN `filename` имя открываемого файла (строка)
- IN `amode` тип доступа к файлу (целое)
- IN `info` информационный объект (дескриптор)
- OUT `fh` новый дескриптор файла (дескриптор)
открывает файл с именем `filename` для всех
процессов из группы коммуникатора `comm`.

Все процессы должны обеспечивать одинаковые значения `amode` и имен файлов, указывающие на один и тот же файл,
`info` используется как «подсказка».

MPI-I/O: Типы доступа

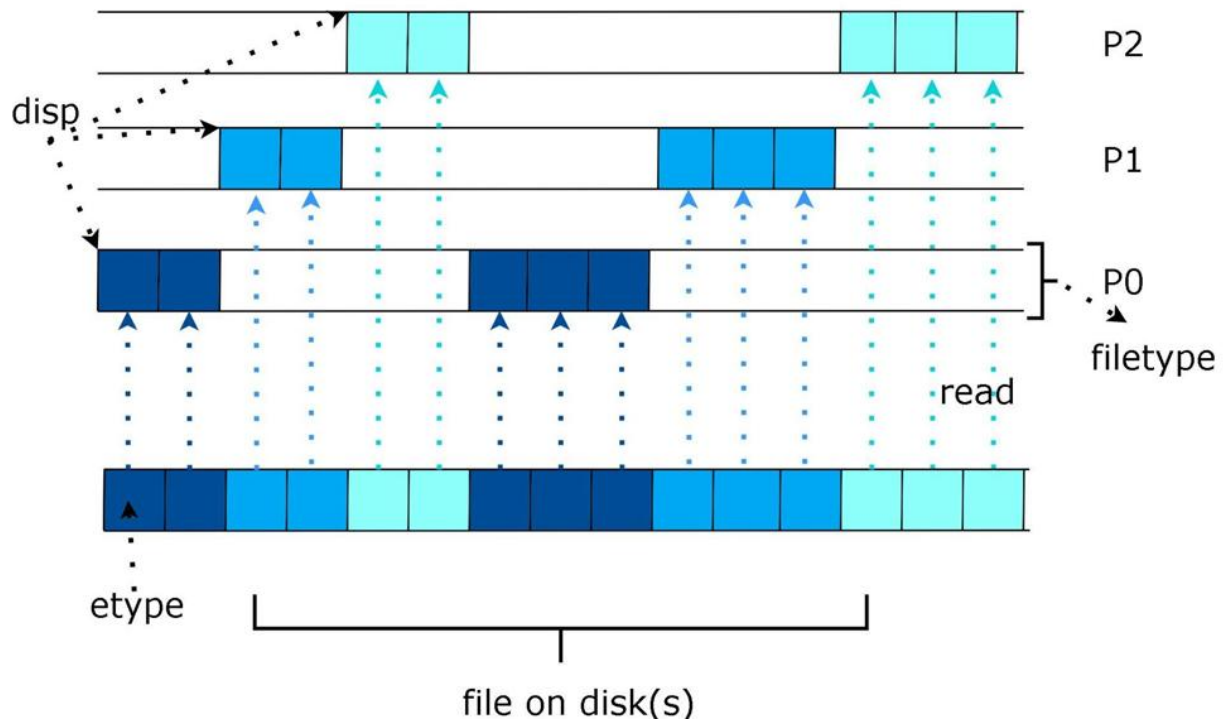
- `MPI_MODE_RDONLY` - только чтение
- `MPI_MODE_RDWR` - чтение и запись
- `MPI_MODE_WRONLY` - только запись
- `MPI_MODE_CREATE` - создавать файл, если он не существует
- `MPI_MODE_EXCL` - ошибка, если создаваемый файл уже существует
- `MPI_MODE_DELETE_ON_CLOSE` - удалять файл при закрытии
- `MPI_MODE_UNIQUE_OPEN` - файл не будет параллельно открыт где-либо еще
- `MPI_MODE_SEQUENTIAL` - файл будет доступен лишь последовательно
- `MPI_MODE_APPEND` - установить начальную позицию всех файловых указателей на конец файла

MPI-I/O: Смещения в файле

```
int MPI_File_set_view(MPI_File fh, MPI_Offset disp,  
MPI_Datatype etype, MPI_Datatype filetype, char  
*datarep, MPI_Info info)
```

- INOUT **fh** дескриптор файла (дескриптор)
- IN **disp** смещение (целое)
- IN **etype** элементарный тип данных (дескриптор)
- IN **filetype** тип файла (дескриптор)
- IN **datarep** представление данных (строка)
- IN **info** информационный объект (дескриптор)

MPI_File_set_view



MPI-I/O: Доступ к данным

```
int MPI_File_read(MPI_File fh, void *buf, int count,  
MPI_Datatype datatype, MPI_Status *status)
```

- INOUT **fh** - дескриптор файла (дескриптор)
- OUT **buf** - начальный адрес буфера (выбор)
- IN **count** - количество элементов в буфере (целое)
- IN **datatype** - тип данных каждого элемента буфера (дескриптор)
- OUT **status** - объект состояния (Status)

MPI-I/O: Доступ к данным

```
int MPI_File_write(MPI_File fh, void *buf, int count,  
MPI_Datatype datatype, MPI_Status *status)
```

- INOUT **fh** - дескриптор файла (дескриптор)
- IN **buf** - начальный адрес буфера (выбор)
- IN **count** - количество элементов в буфере (целое)
- IN **datatype** - тип данных каждого элемента буфера (дескриптор)
- OUT **status** - объект состояния (Status)

MPI-I/O: Доступ к данным

`int MPI_File_read_ordered(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)`

- INOUT `fh` - дескриптор файла (дескриптор)
- OUT `buf` - начальный адрес буфера (выбор)
- IN `count` - количество элементов в буфере (целое)
- IN `datatype` - тип данных каждого элемента буфера (дескриптор)
- OUT `status` - объект состояния (Status)

MPI-I/O: Доступ к данным

`int MPI_File_write_ordered(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)`

- INOUT `fh` - дескриптор файла (дескриптор)
- IN `buf` - начальный адрес буфера (выбор)
- IN `count` - количество элементов в буфере (целое)
- IN `datatype` - тип данных каждого элемента буфера (дескриптор)
- OUT `status` - объект состояния (Status)

MPI-I/O: Заккрытие файла

```
int MPI_File_close(MPI_File *fh)
```

- INOUT `fh` - дескриптор файла (дескриптор)

Сначала синхронизирует состояние файла, затем закрывает файл, ассоциированный с `fh`

Пользователь должен обеспечить условие, чтобы все ожидающие обработки неблокирующие запросы и разделенные коллективные операции над `fh`, производимые процессом, были выполнены до вызова `MPI_FILE_CLOSE`

MPI-I/O: Пример № 1

В файле [writechar.c](#) показан пример записи файла

// Writing file

```
MPI_File_open(MPI_COMM_WORLD,  
              "testfile",MPI_MODE_CREATE |  
              MPI_MODE_WRONLY,MPI_INFO_NULL,  
              &thefile);  
MPI_File_set_view(thefile, myrank * strlen(buf) *  
                  sizeof(char), MPI_CHAR, MPI_CHAR, "native",  
                  MPI_INFO_NULL);  
MPI_File_write(thefile, buf, strlen(buf),  
               MPI_CHAR,MPI_STATUS_IGNORE);  
MPI_File_close(&thefile);
```


MPI-I/O: Пример № 2,3

В файлах [writefile.c](#) и [readfile.c](#) находится пример записи и чтения файла в бинарном виде.

```
//Open file, ordered writing and closing
```

```
MPI_File arr;  
MPI_File_open(comm, "array", MPI_MODE_CREATE |  
MPI_MODE_WRONLY,  
MPI_INFO_NULL, &arr);  
MPI_File_write_ordered(arr, a1, chunks[rank], MPI_DOUBLE,  
MPI_STATUS_IGNORE);  
MPI_File_close(&arr);
```

MPI-I/O: Пример № 2,3

// Reading file

```
MPI_File_set_view(thefile, myrank * bufsize * sizeof(double),  
MPI_DOUBLE,  
MPI_DOUBLE, "native", MPI_INFO_NULL);  
MPI_File_read(thefile, buf, bufsize+ost, MPI_DOUBLE,  
&status);  
MPI_File_close(&thefile);
```

Структуры и MPI

При переходе от последовательной программы к параллельной возникает вопрос, а как можно передавать с помощью средств MPI такие сущности как структуры?

Ответов на этот вопрос два:

- использование передачи побайтово;
- создать свой тип данных в MPI.

Структуры и MPI: побайтовая передача данных

В данном контексте выполняются обычные операции передачи сообщений, в которых в качестве типа передаваемого значения указывается `MPI_BYTE`.

```
MPI_Bcast(&lada,sizeof(car),MPI_BYTE,0,MPI_COM M_WORLD);
```

Пример № 4

В файле `struct_mpi.c` показан пример пересылки структуры как последовательности байтов.

```
//Sending structure as array of bytes  
MPI_Bcast(&lada,sizeof(car),MPI_BYTE,0,MPI  
_CO MM_WORLD);
```

Структуры и MPI: объявление нового типа данных

Передачу структуры также можно осуществить через объявление нового типа данных MPI, совпадающего по составу с определенной ранее структурой.

```
MPI_Type_create_struct(nitems, blocklengths,  
offsets, types, &mpi_car_type);
```

Пример № 5

В файле struct_mpi2.c показан пример создания нового типа данных в MPI и передачи структуры с его помощью.

```
/* create a type for struct car */  
int nitems=2;  
int blocklengths[2] = {1,1};  
MPI_Datatype types[2] = {MPI_INT, MPI_INT};  
MPI_Datatype mpi_car_type;  
MPI_Aint offsets[2];  
offsets[0] = offsetof(car, shifts);  
offsets[1] = offsetof(car, topSpeed);  
MPI_Type_create_struct(nitems, blocklengths, offsets,  
types, &mpi_car_type);  
MPI_Type_commit(&mpi_car_type);  
MPI_Bcast(&lada,1,mpi_car_type,0,MPI_COMM_WORLD);
```

Список литературы

<http://www.mpi-forum.org/> -Message Passing Interface Forum. Сайт-форум, посвященный стандарту MPI. Содержит полную документацию по MPI.

<http://www.netlib.org/utk/papers/mpi-book/mpi-book.html>

MPI: The complete Reference. MIT Press, Cambridge, Massachusetts, 1997. Авторы: Marc Snir, Steve Otto, Steve Huss -Lederman, David Walker, Jack Dongarra.

<http://www.open-mpi.org/> - Open MPI—свободно распространяемая реализация MPI и среда разработки MPI-программ для гетерогенных кластеров из UNIX-машин.

Список литературы

<http://www.parallel.ru> -сайт лаборатории
Параллельных информационных
технологий МГУ.

<http://parallel.ru/docs/mpich/html/> - список MPI-
команд.

<http://hybrilit.jinr.ru/> - Гетерогенный кластер
HybriLIT.

<https://computing.llnl.gov/tutorials/mpi/> -Message
Passing Interface (MPI).

В.В.Воеводин, Вл.В.Воеводин "Параллельные
вычисления", БХВ-Петербург, 2002.

Файлы и структуры в MPI